Lab 03

Tổng quan về Cấu trúc dữ liệu & Giải thuật

A. Chuẩn bị

- Đọc kỹ slide bài giảng
- Tài liệu tham khảo [1] Robert Sedgewick. (1998). *Algorithms in C, Part 1-4, Third Edition*. Addison-Wesley

B. Hướng dẫn

1. Tạo thư mục CTDL-BTTH-TuanX-MSSV (với XX là tuần học, ví dụ Tuan1, Tuan2,…)
2. Làm bài & Lưu tất cả file bài làm vào thư mục trên.
3. Xóa các file exe → Nén thư mục này → file nén CTDL-BTTH-TuanX-MSSV.zip
4. Nộp file nén CTDL-BTTH-TuanX-MSSV.zip vào mục "Bài tập thực hành tuần X" trên mLearning.

C. Thực hành

Bài tập 1.

Cài đặt các thuật toán tham lam (bài toán đổi tiền), thuật toán chia để trị (bài toán tìm đồng xu giả).

Hint: tham khảo slide bài giảng.

Bài tập 2.

Cài đặt thuật toán đếm các cặp số chia hết cho 5 trong mảng n số nguyên ngẫu nhiên.

Tính thời gian thực hiện trên máy nếu n = 5000, n = 10000, n = 25000, n= 50000

Hint:

```c
// Đếm các cặp phần tử có tổng chia hết cho 5
// Bản chất như bài toán đếm các cặp đôi "hợp" tuổi :)
// Độ phức tạp: O(n^2)
int count_pairs(int a[], int n) {           // block A: 1 lần
    int count = 0;
    for (int i = 0; i < n; i++) {           // block B: N
        for (int j = i + 1; j < n; j++) {   // block C: N * (N - 1) / 2 ~ N^2
            if ((a[i] + a[j]) % 5 == 0) {
                count++;                    // block D: x lần (tùy thuộc input)
            }
        }
    }
    return count;
}
```

```c
#include <stdio.h>
#include <stdlib.h>
#include <time.h>

void randomArray(int a[], int n) {
    // Bộ phát sinh ngẫu nhiên

    srand(time(0));
    // Phat sinh n so ngau nhien trong khoang [0, MAX)
    int MAX = n - 1;
    int i, value;
    for (i = 0; i < n; i++) {
        value = rand() % MAX;
        a[i] = value;;
    }
}
int main() {
    int n = 100000;
    int a[n];
    float start, end;
    randomArray(a, n);
    start = clock();
    // Tính toán...

    // Kết thúc tính toán
    end = clock();

    return 0;
}
```

Bài tập 3.

Xác định số thao tác tính toán trong thuật toán tìm giá trị lớn nhất trong mảng các số nguyên trong trường hợp tốt nhất (best case), xấu nhất (worst case), trung bình (average case)

Bài tập 4.

Xác định số thao tác tính toán trong thuật toán tìm một số x (có xuất hiện hay không?) trong mảng các số nguyên trong trường hợp tốt nhất (best case), xấu nhất (worst case), trung bình (average case)

Bài tập 5.

Viết chương trình xác định 4 số nguyên dương có khác biệt hay không.

Trường hợp tốt nhất (best case) và xấu nhất (worst case) cần bao nhiêu phép toán/thao tác so sánh?

Ví dụ: 5, 8, 2, 7 là các số nguyên khác biệt

Bài tập 6.

Xác định độ phức tạp về thời gian T(n) của thuật toán đệ quy tính lũy thừa a^n

```
double recursive_power(double a, int n) {
    if (n == 0) {
        return 1;
    }
    return a * power(a, n - 1);
}

```

Bài tập 7.

Xác định độ phức tạp về thời gian T(n) của thuật toán lũy thừa a^n

```
double power(double a, int n) {
    if (n == 0) {
        return 1;
    }
    if (n % 2 == 0) {
        double half = power(a, n / 2);
        return half * half;
    } else {
        return a * power(a, n - 1);
    }
}
```

Bài tập 8.

Viết chương trình tính giá trị trung bình (mean value) của mảng các số nguyên dương a[], và đếm số phần tử trong mảng lớn hơn giá trị trung bình.

Phân tích chương trình: độ phức tạp về thời gian (time complexity) T(n) trong các trường hợp tốt nhất, xấu nhất là bao nhiêu? (xác định Big O).